Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates

Leslie N. Smith, Nicholay Topin. arXiv:1708.07120v3, 2018

The paper's main question

- Common concern: a large SGD step may make training unstable or cause it to diverge.
- What authors observed: raising the learning rate and then lowering it can give good accuracy in far fewer iterations.

The authors call this unusually fast training super-convergence.

The usual learning rate schedule

- Piecewise constant LR: the learning rate stays near 0.1 for a long time. When progress stalls, it is divided by ten.
- Why it is slow: several long phases may be needed before the model reaches its final accuracy.

The paper tests whether a large LR used early can shorten training.

Cyclical learning rates

Phase	Learning rate	Length
First half	Linear rise from min to max	stepsize iterations
Second half	Linear fall from max to min	stepsize iterations

One CLR cycle has two equal length phases.

Min, max, and stepsize must be chosen before training.

Choosing the learning rate bounds

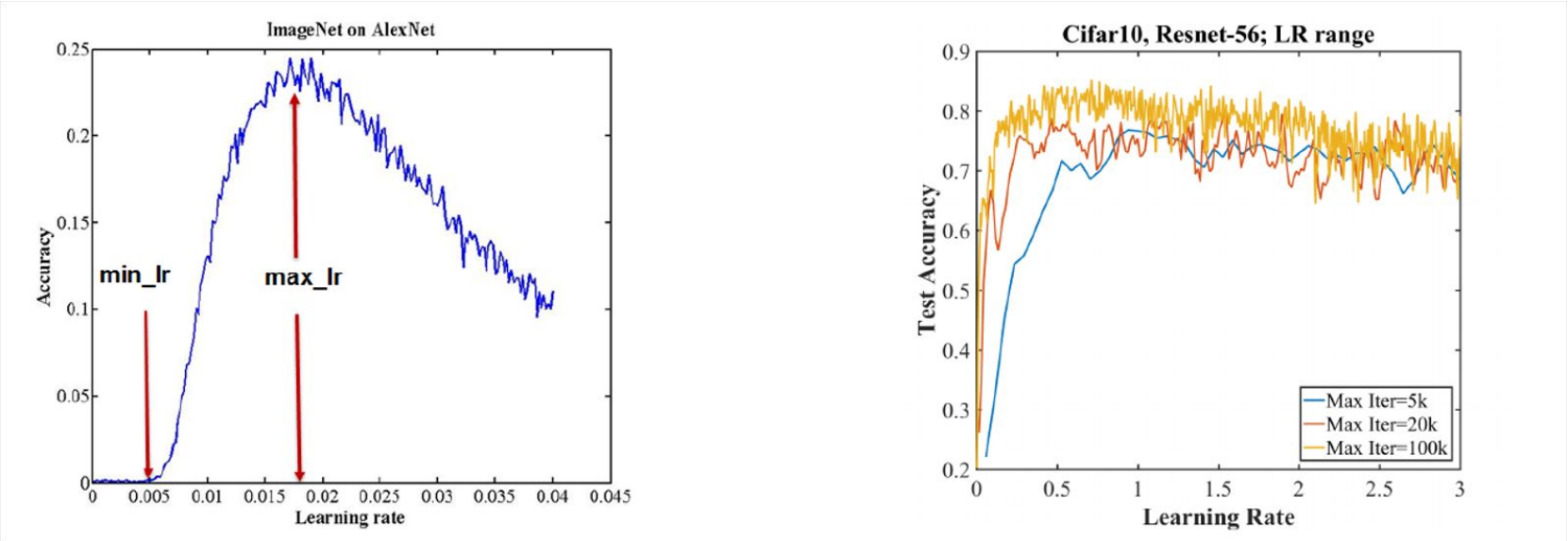


Figure 2: a clear peak on the left; ResNet-56 remains accurate up to LR 3.0 on the right.

Set max near the peak and min roughly three to four times lower.

The 1cycle schedule

Phase	Learning rate	Purpose
1	Rise from min to max	Fast progress and strong regularization
2	Fall from max to min	Move toward a more precise solution
3	Fall below the initial rate	Final stabilization

1cycle means one main cycle with a final low LR phase.

Momentum and other settings

- Momentum: it usually falls as LR rises, then returns while LR falls.
- MNIST example: LeNet, 12 epochs: LR 0.01-0.1, momentum 0.95-0.8, stepsize 5 epochs

LR, momentum, weight decay, and batch size affect each other..

Why a large LR may help

- Authors explanation: large parameter updates may keep training away from narrow minima.
- Practiccal consequence: increasing LR can require less weight decay or other regularization.

A larger LR is useful only while training remains stable.

Estimating LR from curvature

Expression for one parameter

eps* = eps * (theta_(i+1) - theta_i) / (2 theta_(i+1) - theta_i - theta_(i+2))

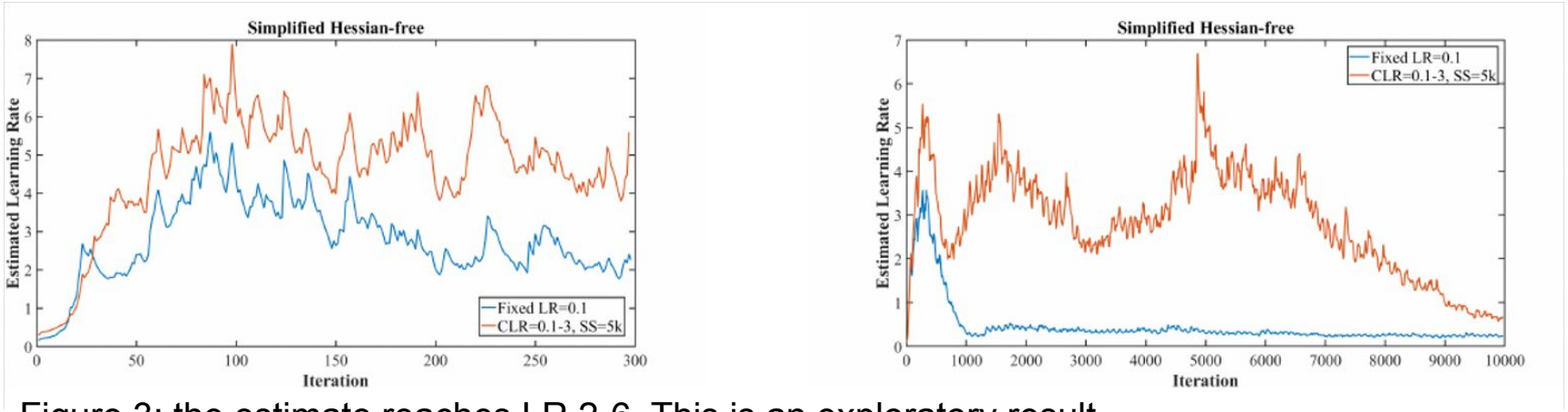


Figure 3: the estimate reaches LR 2-6. This is an exploratory result.

Main result: CIFAR-10 and ResNet-56

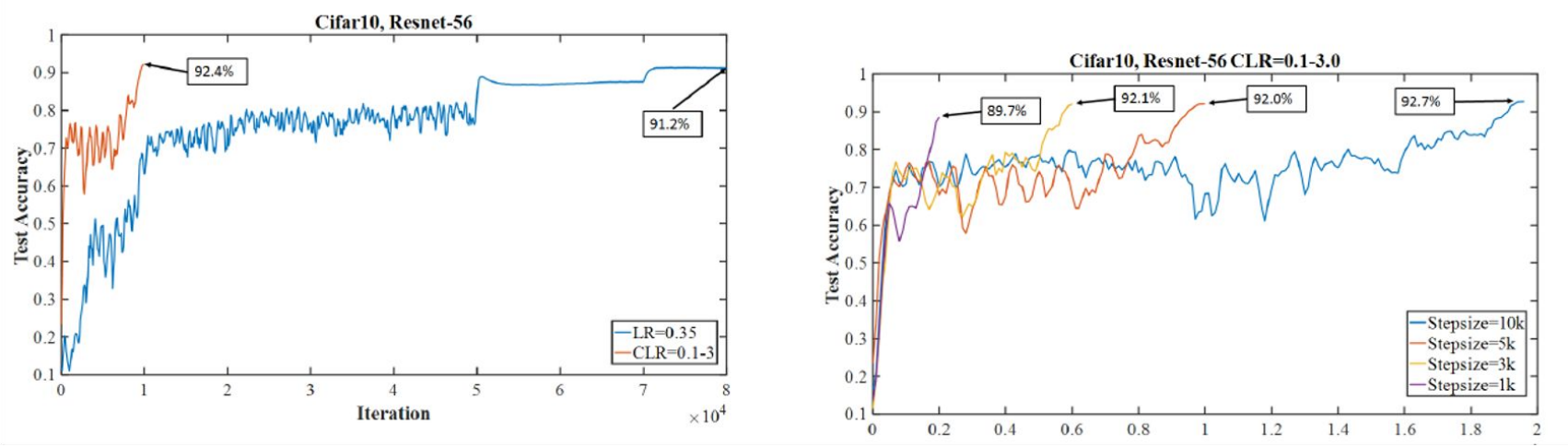


Figure 1: standard training reached 91.2% in 80,000 steps; CLR reached 92.4% in 10,000.

This comparison uses eight times fewer optimizer steps.

Results with less training data

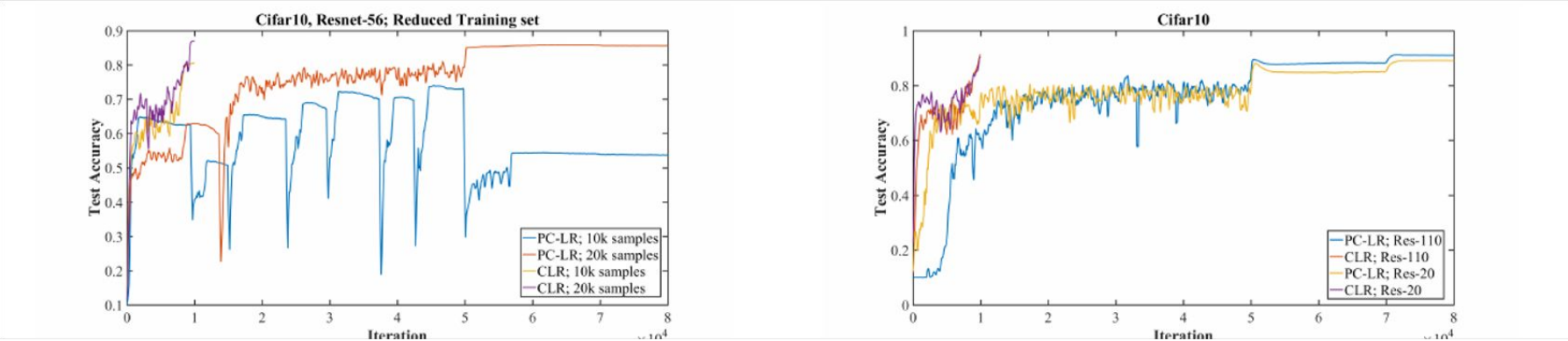


Figure 4: the gap is especially clear with 10,000-20,000 images.

The next slide shows the exact values and a change in the BN setting.

Table 1: limited training data

# training samples	Policy (Range)	BN MAF	Total Iterations	Accuracy (%)
40,000	PC-LR=0.35	0.999	80,000	89.1
40,000	CLR (0.1-3)	0.95	10,000	91.1
30,000	PC-LR=0.35	0.999	80,000	85.7
30,000	CLR (0.1-3)	0.95	10,000	89.6
20,000	PC-LR=0.35	0.999	80,000	82.7
20,000	CLR (0.1-3)	0.95	10,000	87.9
10,000	PC-LR=0.35	0.999	80,000	71.4
10,000	CLR (0.1-3)	0.95	10,000	80.6

At 10,000 images: 71.4% versus 80.6%, a gap of 9.2 percentage points.

BN MAF also changes: 0.999 for PC-LR and 0.95 for CLR.

Other CIFAR-10 architectures

Architecture	Standard schedule	CLR / 1cycle
ResNet-20	88.6%	90.4%
ResNet-110	91.0%	92.1%
Wide ResNet	90.3%, 800 epochs	91.9%, 100 epochs
DenseNet	92.7%, 400 epochs	92.8%, 150 epochs

Several networks benefit, but the size of the gain varies.

CIFAR-100: accuracy by training length

Schedule	Epochs	Accuracy
Standard	200	61.6%
1cycle	25	65.4%
1cycle	50	66.4%
1cycle	100	69.0%

Even 25 epochs of 1cycle beat 200 epochs of standard training in this table.

MNIST and LeNet: Table 2

Schedule	Epochs	Accuracy
Inverse	85	99.03%
Step	85	99.00%
1cycle	12	99.25%
1cycle	25	99.28%
1cycle	50	99.27%
1cycle, LR 0.02-0.2	85	99.35%

For 12 epochs: LR 0.01-0.1, momentum 0.95-0.8, stepsize 5 epochs.

ImageNet: larger models

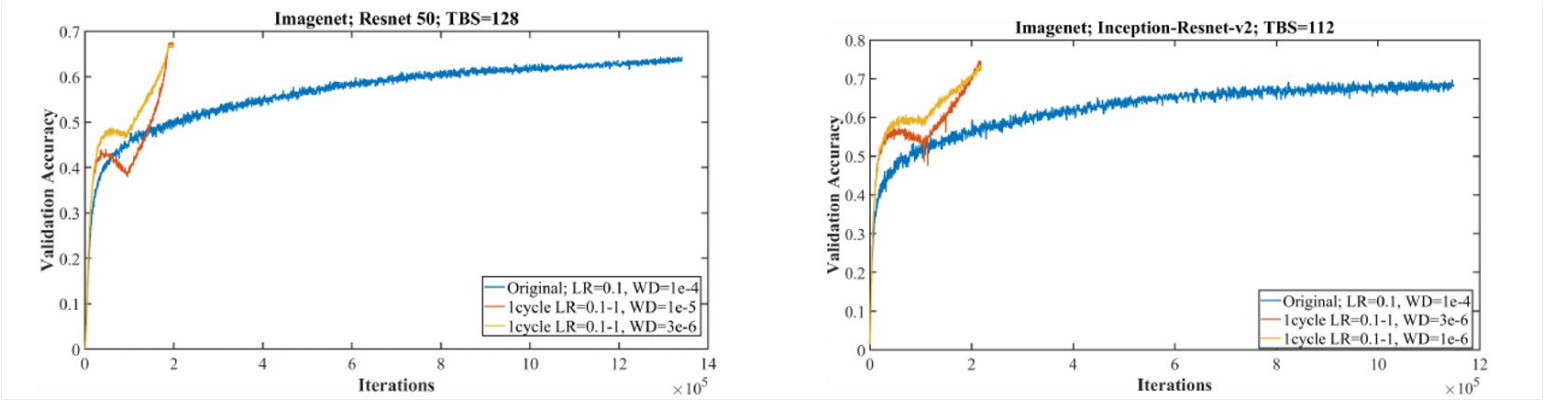


Figure 6: ResNet-50, 63.7% vs 67.6%; Inception-ResNet-v2, 67.6% vs 74.0%.

Weight decay also changed. The curves average two runs.

Batch size and batch normalization

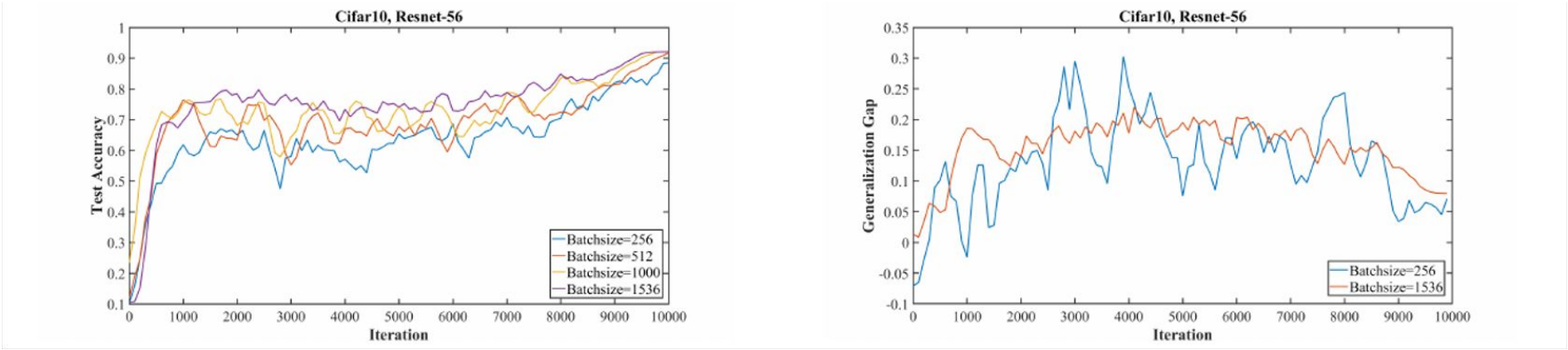


Figure 5: larger batches worked better under these settings.

The authors also reduced BN moving_average_fraction from 0.999 to 0.95.

Takeaways from the paper

1. Run an LR range test and choose the bounds.
2. Use one rise-and-fall LR cycle.
3. Finish training and a very small LR.
4. Tune momentum, weight decay, and BN statistics.
5. Compare accuracy and training time with a baseline.

Main idea: a large LR can speed up training and act as a regularization.